



Color gradient vectorization for SVG compression of comic image[☆]



Ray-I. Chang, Chung-Yuan Su^{*}

Department of Engineering Science and Ocean Engineering, National Taiwan University, 1, Roosevelt Road, Sec. 4, Taipei 106, Taiwan, ROC

ARTICLE INFO

Article history:

Received 17 October 2014

Accepted 14 September 2015

Available online 21 September 2015

Keywords:

SVG

Vector image

Comic compression

Color gradient

Color gradient vectorization

Color clustering

Improved vector contour searching

Middle point detection

ABSTRACT

In past years, different raster-to-vector methods were proposed to convert bitmap images to SVG format. However, they did not consider the color gradient (CG) that frequently appears in the comic image. Their results needed multiple divided color regions to represent a single CG region. It produced poor perceptual quality and large SVG size. In this paper, we propose the CGV (CG vectorization) method to resolve this problem. CGV first applies a linear-time algorithm to identify the CG vector for representing the color and the direction of CG in each region. Then, we merge neighboring regions those have the same CG vector as a large CG region and represent it by a single path of SVG with linear gradient syntax. Experimental results show that our method outperforms other state-of-the-art SVG vectorization systems in terms of not only SVG size but also perceptual quality. For example, the averages SSIM are 0.85 by Autotrace, 0.92 by Autotrace+Merge, 0.88 by SWaterG, 0.91 by Vector Magic and 0.94 by our method. It is about 57.23% for our average space saving. Moreover, comparing with other systems' results, our SVG files take the shortest time (about 0.12 s in average) of rendering on handheld devices.

© 2015 Elsevier Inc. All rights reserved.

1. Introduction

Comic (or manga) is an art form using drawn images to tell a story. It is often printed in newspapers first, and may be collected as a comic book. With the popularity of handheld devices, readers' habit has a significant and steady transition from printed to digital. The growth of E-book market is fast [3]. Many companies (such as Marvel Comics [2]) are going to publish comics in digital form. As most of the comic data are still in raster formats (such as JPEG and BMP), converting these comics from printed to digital needs to face the multiple-resolution problem for fitting different display devices. Conventionally, the pixel interpolation method is applied to rescale the image. It causes the degradation of image quality (such as jagged or fuzzy edges). Vector formats, such as SWF and SVG (scalable vector graphics) [4], provide an alternative way to compensate the mentioned drawbacks. In addition, a vector format file can easily embed the semantic meaning of comic by text for modern search engines. In this paper, we apply SVG (the current version is 1.1) in raster-to-vector conversion as it is a graphic standard of W3C. It is also the standard format for 2D graphics in HTML5 and has further been used in the context of EPUB

publications. For supporting mobile devices, SVG has provided a lightweight version called "SVG Tiny" (SVG-T, the current version is 1.2). As SVG has many good characteristics (such as arbitrarily scalable, easily editable, being indexed by search engines, and seamless integration with the ever-expanding XML), several raster-to-vector methods [1,5,6,8,15] have been proposed to convert bitmap images to SVG format. Nowadays, the business potential of SVG has been discovered, and some commercial systems (such as Stanford Vector Magic [9], Adobe Illustrator and CorelDraw) are provided. However, they did not consider the color gradient (CG) problem in comic images. In [1], a raster-to-vector system was proposed with higher compression ratio and better perceptual quality than previous methods. We improved [1] in this paper to propose the CGV (CG vectorization) method to identify CG regions and apply the linear gradient syntax of SVG to represent these regions. In this paper, without loss of generality, we use 130 comic images collected from Internet as our test data. In the quality measurement, the averages SSIM are 0.85 by Autotrace, 0.92 by Autotrace+Merge, 0.88 by SWaterG, 0.91 by Vector Magic and 0.94 by our method. In size measurement, our method can save about 57.23% for average space saving. Our average rendering time is about 0.12 s. Both space saving and rendering time are much better than the other methods.

Comic (or cartoon) images are very suitable for representing in vector format because of their geometry elements and color regions. Comic writers draw and fill regions with CG to make the comics look more colorful and real, as shown in Fig. 1. However, conventional raster-to-vector systems did not consider the CG

[☆] This work was partially supported by NSC Taiwan under grant 102-2410-H-002-170-MY3. The authors wish to express the appreciation to Mr. Po-Yen Pan for his help with the experiments.

^{*} Corresponding author.

E-mail addresses: rayichang@ntu.edu.tw (R.-I. Chang), d98525007@ntu.edu.tw (C.-Y. Su).



Fig. 1. A color gradient comic image and its conventional vectorization result are shown. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

region and typically separated it into different regions with flat colors. Three examples are shown in the bottom of Fig. 1. As there are sudden changes of colors in different regions, the image quality is degraded and causes a large SVG file. Moreover, rendering a large SVG file is always a big challenge for handheld devices. In this paper, we propose a novel method named CGV (CG vectorization) to solve this problem. Note that, in a CG region, the color usually changes from one color to another in a straight line. Based on this observation, CGV first identifies the CG vector (denoted as “path” in SVG) for representing the color and the direction of CG in each region. Then, it merges the neighboring regions those have the same CG vector. At last, this merged region is represented by a single path of SVG with linear gradient syntax. Experimental results show that CGV substantially outperforms other state-of-the-art vectorization methods in terms of both file size and perceptual quality. Moreover, comparing with other systems’ results, our SVG files take the shortest time of rendering on handheld devices.

We summarize the contributions of this paper as follows. (1) The proposed method can identify CG in the comic image and embed it in SVG syntax. It provides not only good perceptual quality but also small file size. (2) The proposed method can remove redundant points in SVG paths. It improves the compression ratio and reduces the rendering time. (3) The proposed method can utilize the reusability of SVG syntax. It further minimizes the SVG file size. The rest of this paper is organized as follows. In Section 2, we present an overview of existing raster-to-vector approaches where their disadvantages led us to develop our method. Next, we describe our system framework and methods including their implementation details in Section 3. In Section 4, we perform several experiments on real comic images to validate the proposed system. Conclusion and future works are given in Section 5.

2. Related works

As the continuous tone in comic may be printed as halftone dots. Kawamura et al. [6,7] first proposed the tone detection method to reconstruct continuous tone for improving the image quality of comic. Šýkora et al. [13,14] presented the color-by-example technique (that combines image segmentation, patch-based sampling, and probabilistic reasoning) to enhance the perceptual quality of comic images. Qu et al. [12] proposed the user-intervention system to measure the pattern-continuity of comic images. It used a local, statistical based pattern feature obtained with Gabor wavelet filters. The boundary is propagated by the level set method that monitors the pattern continuity when the user scribbles on the drawing in comics. However, these three

methods work only on black-and-white images, and their output files [13,14,12] were still in raster format.

Note that, although conventional raster-to-vector methods can convert bitmap images to SVG format, the results have the drawback of large file size. In past years, different approaches were proposed to process (compress) the vector image. The main idea of vector image compression is to delete or merge surplus objects in the SVG file to reduce its file size. W3C [18] and Adobe [10] have brought up some SVG “simplifying” methods to remove useless SVG information. They also tried to simplify control units for the speedup in image rendering. However, the resulted compression ratio was not good enough. Battiatto et al. [8] applied DDT (Data Dependent Triangulation) to approximate local pixel neighborhood. It replaced the input image with a set of triangles. Then, each triangle was remapped into a vector format SVG representation. Adjacent triangles those have the similar filling colors are merged to reduce the SVG file size. As these triangles are according to a specific cost function for implicitly detecting the edge details, the resulted image quality is poor. In [15], the input image was partitioned into homogeneous and contiguous regions using watershed decomposition. Then, each region is represented by the ad-hoc chain code. As long as you can merge many regions to form a single region, the file size of SVG image is reduced. As its vectorization result has kept all bitmap information of the image, it generated a large SVG file. In [1], we adopt Autotrace [11] to raster-to-vector processing of comic images. Our system proposed the vector contour searching (VCS) algorithm for removing extra spaces, combining the slope of clips, and merging similar color regions. By compressing the vector image, our system can keep good image quality and achieve high compression ratio. However, it did not consider the CG problem in the comic image. Among existing SVG gradient technologies, Zhang et al. [5] proposed a new trapped-ball method combined with detected decorative lines to segment comic image. Temporal coherence is used to extract a unified background or foreground before vectorization to achieve flicker-free animation. Their method targets animation included smoothly colored regions and decorative lines. As previous approaches, this method tried to reconstruct regions with smoothly change color to produce visually acceptable results. The generated SVG file is large. Jian et al. [22] presented a semi-automatically tool to create an optimized gradient mesh for image vectorization. First images are separated into different objects; then each object is modeled by using an optimized gradient mesh (Coons patch mesh). Lecot and Levy [23] created a vector image that approximates a bitmap image named ArDeco. Their segmentation method divides image into triangulation (trixels) as regions. Each region is then approximated by a mesh with linear or radial gradients to minimize Mumford and Shah’s energy functional. Although their approaches can represent a large variety of image objects containing smooth transitions, the vectorization results they generated are large. Kansal and Kumar [24] performed the range of gradient values by using the best-fit method after segmentation. They divided each region according to the error for a segment to optimize gradient. Although they reconstructed the linear gradient by estimating gradient stops and direction, they did not explain the size of SVG in detail. This drawback motivates us to improve our previous algorithm to embed CG for providing better perceptual quality and file size.

3. The proposed CGV method

In this paper, we improve our previous method and implement a new raster-to-vector system called CGV (CG vectorization) to consider CG that frequently appears in comic images. Fig. 2 shows the framework of our system. Input a raster format comic image,

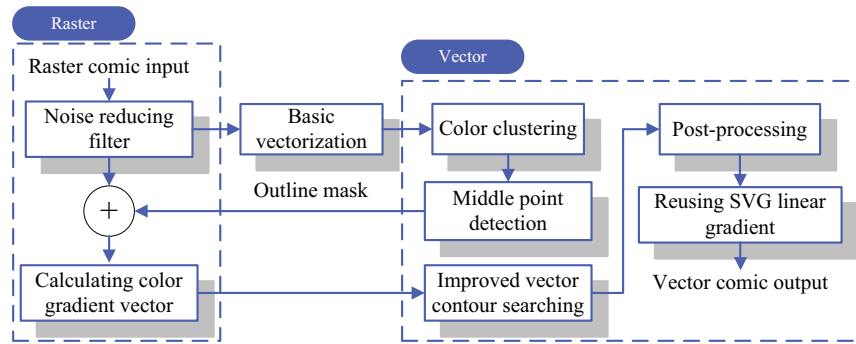


Fig. 2. Our system framework for raster-to-vector with color gradient. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the noise-reducing filter is applied to reduce the visual noises. Then, the basic Autotrace vectorization process is processed. After doing the basic vectorization, we apply the color clustering method to merge multiple small regions with the similar colors as a single region. As the number of regions is reduced, we have a smaller SVG file size and the computation time required in the next phase can be also reduced. In this paper, an improved VCS (IVCS) algorithm is proposed. We first identify points (called “middle point”) those are the intersection points of two neighboring regions (paths) but only appear on one of them. As this vacancy will make a bad decision in IVCS to merge neighboring regions, we will fill the vacancy before doing IVCS. Note that each path of the region can form an outline mask to calculate its CG vector (which contains its color and gradient-direction). As IVCS, the neighboring regions with the same CG vector can be merged as a large CG region to reduce the SVG file size. Then, we propose the post processing method to eliminate redundancy points of paths. At last, reused the SVG linear gradient will be applied in regions with similar gradient. The detail operations of our procedures are described in the following sub-sections.

3.1. Noise-reducing filter and autotrace vectorization

As current comic images are usually in JPG format, they have annoying visual noises (such as ringing-effect) around edges. These noises may cause many false edges when doing vectorization by Autotrace. Therefore, in some cases, the produced SVG file may be larger than its input JPG file. In this paper, we apply the well-known bilateral filter [16] for noise-reducing as it is quite effective in resolving ringing-effect and preserving the real edges. Then, we choose Autotrace for basic vectorization since it has been applied in many previous researches. In this paper, we use the default setting of Autotrace for fair comparisons. As shown in Fig. 3(a), Autotrace preserves the details of the comic image. Unfortunately, it

may produce not only many small and scrappy areas around the main area, but also the snow effect in the rendering result. For resolving these problems, color clustering and middle point detection schemes are proposed.

3.2. Color clustering and middle point detection

Our color clustering algorithm first sorts all the regions in descending order of their size. Then, we take the largest region as the first cluster. When a region is neighboring to a cluster, we calculate the color distance value ΔE between this region and the neighboring cluster. When ΔE is smaller than a threshold TH , we update the color of this cluster by the average color of the region and this cluster. Otherwise, the region is considered as a new cluster. The algorithm are terminated until all the regions are clustered. In this paper, we choose the CIE $L^*a^*b^*$ as our color space and define $TH = 6$ for the upper limit of color difference that human can recognized [17]. Give the SVG file shown in Fig. 3(a), the result of our color clustering scheme is shown in Fig. 3(b).

After doing Autotrace, we observe a drawback of some intersection points of two paths (regions) only exist in one path. This kind of vacancy called middle points in this paper may cause errors in searching contour and produce snow effect. As can be seen in Fig. 3(a), snow effect caused by the presence of gaps between near elements and showed the color of SVG canvas (often white color). In Fig. 4, as points in the same path are shown by the same color, points with two colors are intersection points. Note Autotrace follows counter-clockwise direction to track boundary to achieve a vectorial representation. Based on this characteristic, we all start at point A and then along counter-clockwise way back to point A by executing our middle point detection. For example, when we track to point C, this point exists in both paths and should be an intersection point. However, point C is identified only in path 2. As shown in Fig. 4, we will setup tolerant search region between

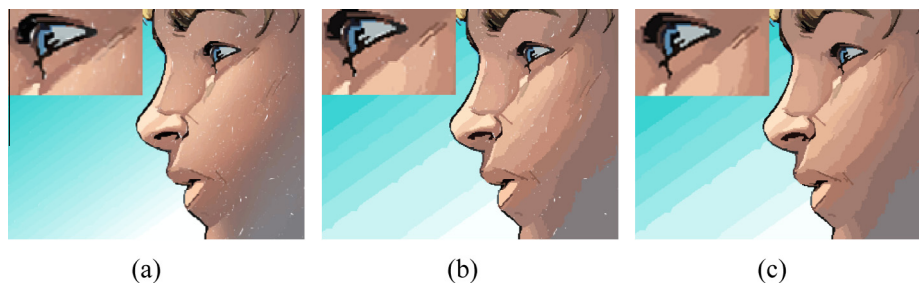


Fig. 3. The result of comic by using: (a) Autotrace, (b) color clustering and (c) middle point detection. Note that Autotrace may produce not only many small and scrappy areas but also the snow effect in the rendering result. By applying our color clustering and middle point detection schemes, the scrappy areas with the similar colors are grouped and the snow noises have been removed. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

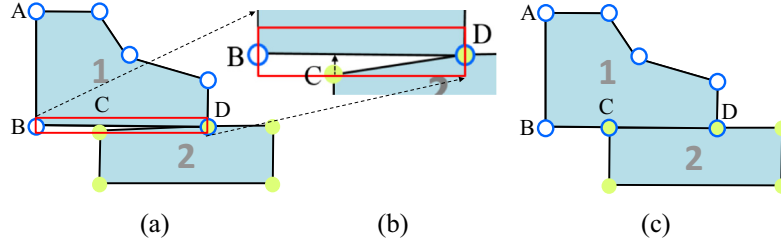


Fig. 4. (a) The example of the middle point. (b) An illustration of the middle point that makes the snow noise. (c) Contour validation after middle point detection.

point B and its next point D to add point C into path 1. This tolerant search region can avoid finding the wrong middle point and then producing the result of over-contour validation. The color clustering result of our middle point detection scheme is shown in Fig. 3 (c). The snow noises have been removed.

3.3. Color gradient vectorization

Previous vectorization methods break the CG of the comic image into many color regions. They usually degrade the visual quality and produce large file size. In this paper, we propose a novel CG vectorization technique called CGV. Our method can identify a directional change in intensity by using the horizontal and vertical derivative approximations (denoted as G_{x_i} and G_{y_i} , respectively) of the comic image. $G_{x_i}(m, n)(G_{y_i}(m, n))$ is calculated by a mask of $M_x(M_y)$ convolved with R_i to calculate horizontal (vertical) change at the pixel (m, n) as follows.

$$M_x = \begin{bmatrix} -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \end{bmatrix} \text{ and } M_y = \begin{bmatrix} -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \end{bmatrix}^T \quad (1)$$

$$G_{x_i}(m, n) = \sum_{k=-3}^3 \sum_{l=-3}^3 M_x(k, l) R_i(m+k, n+l) \quad (2)$$

$$G_{y_i}(m, n) = \sum_{k=-3}^3 \sum_{l=-3}^3 M_y(k, l) R_i(m+k, n+l) \quad (3)$$

where R_i represents region i in the comic image. The Eqs. (2) and (3) can be written as below.

$$G_{x_i} = M_x * R_i \text{ and } G_{y_i} = M_y * R_i \quad (4)$$

where “*” denotes the 2-dimensional convolution operation. Then, we can obtain gradient map $GM_{R_i}(\theta)$ by calculating the gradient's direction in the following.

$$GM_{R_i}(\theta) = \tan^{-1} \left(\frac{G_{y_i}}{G_{x_i}} \right) \quad (5)$$

Fig. 5(a) shows the example of gradient vector that indicates in the horizontal direction of intensity increasing from dim area to light

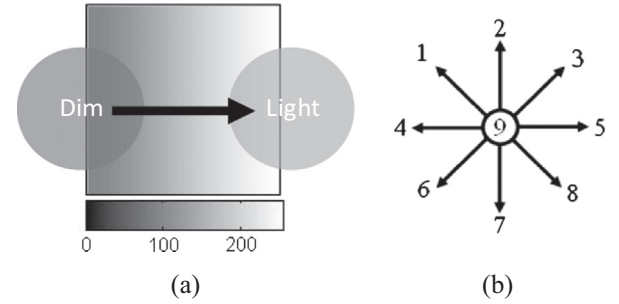


Fig. 5. (a) Linear gradient with a direction of the gradient from dim to light. (b) Eight-direction of gradient applied in this paper.

area. Gradient directions and its corresponding numbers are shown in Fig. 5(b). If there is a region with constant color, the number of gradient direction is 9.

These gradients ($0^\circ, 1^\circ, \dots, 359^\circ$) are grouped into b direction bins and then accumulated over each region R_i to form a probability histogram of gradient directions $P_i(GDs_j)$. In this paper, the number of direction bins is 8; therefore, each bin is 45° width ($360^\circ/b$). We choose the maximum value of directions from probability histogram as this region's CGV.

$$CGV_i = \arg \max_{j=1}^b P_i(GDs_j) \quad (6)$$

Fig. 6(a) shows an example of gradient map from a region with red boundary. Fig. 6(b) shows the probability histogram from this region. After identifying CGV from original comic images, the IVCS method is then used to merge paths with similar CG vectors as shown in Fig. 6(c).

3.4. Improved vector contour searching (IVCS)

In [1], we propose VCS that performs well for searching regions with simple and uniform colors. In first round, we follow quadrant decision (e.g. $\min X$, $\min Y$, $\max X$, $\max Y$) to track points that are the outmost points as candidate points when meeting intersection points. Then the central point is calculated from contour searching results. To validate the contour, anchor decision is used to calculate the distance between a central point and candidate points, and the point with the largest distance is the contour point. More details can be found in [1]. However, VCS produces bad results for finding outer contour with CG due to it applies the quadrant decision to find the error candidates as an outer contour. This will also lead our outermost contour to failure by applying the Anchor decision. In this paper, we improve VCS to identify and merge CG regions. The proposed algorithm tracks points in counter-clockwise direction to search the outer contour. If the current point appears only on one path, the next point of contour would be just the next point of the current path. However, if it appears on more than one path (such as the intersection point), we should consider four cases to

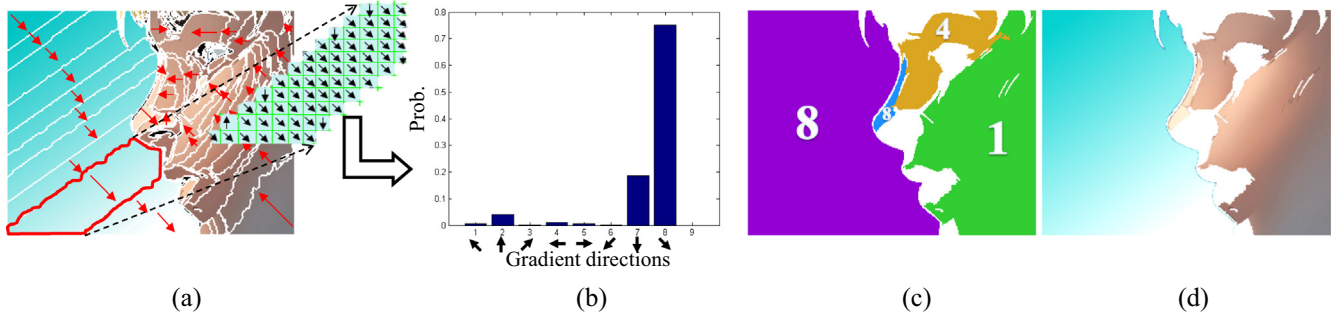


Fig. 6. The results of CGV and linear gradient approximation in the comic image. (a) The CGV is identified by choosing the maximum value of gradient directions from its probability histogram. (b) The probability histogram from the region with red boundary. (c) The result of IVCS is to merge paths with similar CG vector. (d) The result of SVG gradient approximation from original comic image by embedding the linear gradient syntax. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

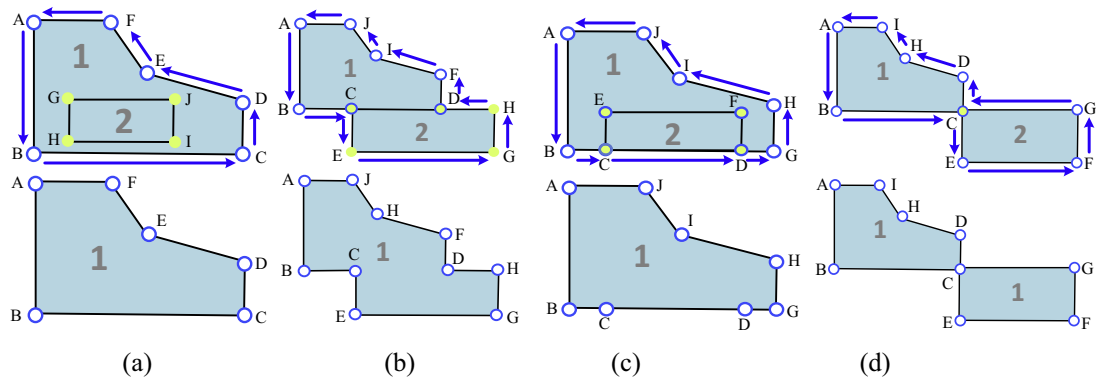


Fig. 7. Four cases of our IVCS scheme. (a) Case 1. (b) Case 2. (c) Case 3. (d) Case 4.

merge neighboring paths. According to the characteristic of SVG data, the searching route tracks outmost contour by the blue arrows and starts from and goes back to the same point A.

Case 1 (The intersection points = 0 and the $path_i$ are all enclosed by the $path_j$):

In Fig. 7(a), the points in path 2 (G, H, I, J) are all enclosed by the path 1; therefore, we start from point A and find the next point B without perplexity. After tracking back to point A ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow A$), two paths can be merged as shown in Fig. 7(a) (bottom).

Case 2 (The intersection points > 1 and the points of $path_i$ are not all enclosed by the $path_j$):

Fig. 7(b) shows that point C appears in both two paths. There are two possible candidates (D and E) followed by the point C ($C \rightarrow D$, $C \rightarrow E$). We choose point E as it appears only in one path. When tracks to point D, we use the same rule to choose point F as the next point. As shown in Fig. 7(b) (bottom), two paths are merged when the IVCS procedure tracks back to point A ($A \rightarrow B \rightarrow C \rightarrow E \rightarrow G \rightarrow H \rightarrow D \rightarrow F \rightarrow I \rightarrow J \rightarrow A$).

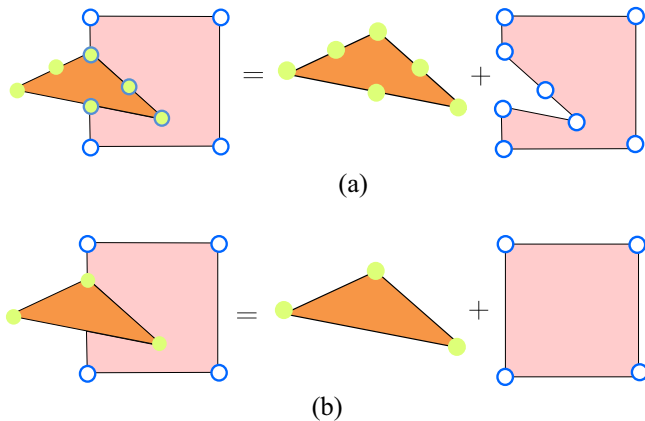


Fig. 8. An example of concave and overlapped paths for compression in SVG file. (a) Not optimal result. (b) Optimal result.

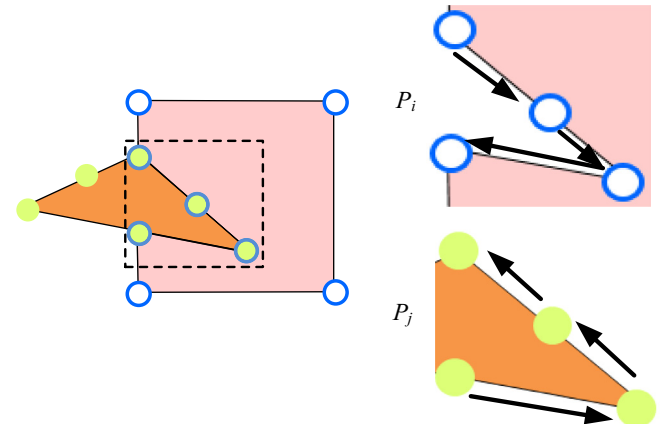


Fig. 9. The concept of path simplifications by applying data characteristics to identify layers' order.

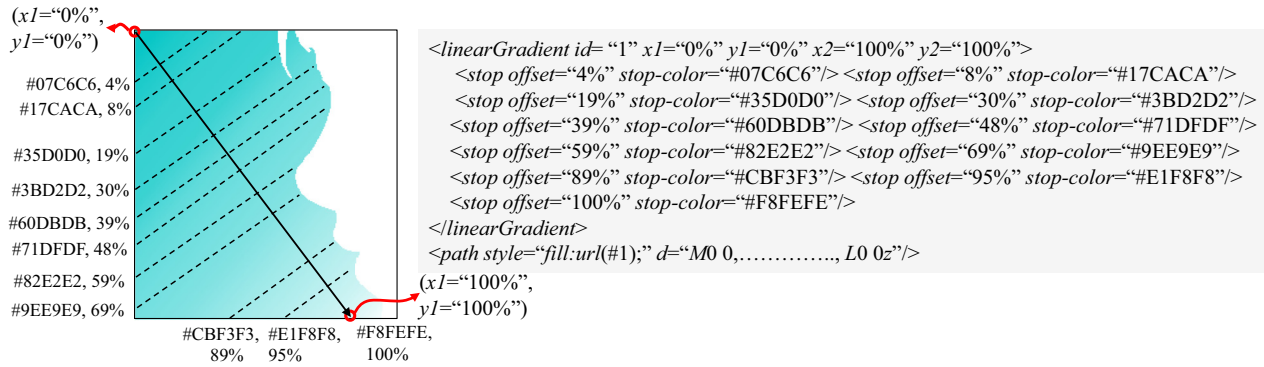


Fig. 10. The attributes and the linear gradient syntax. Note that this image is a part of Fig. 6(d), and the syntax of path is not completely provided since there are too many coordinates.

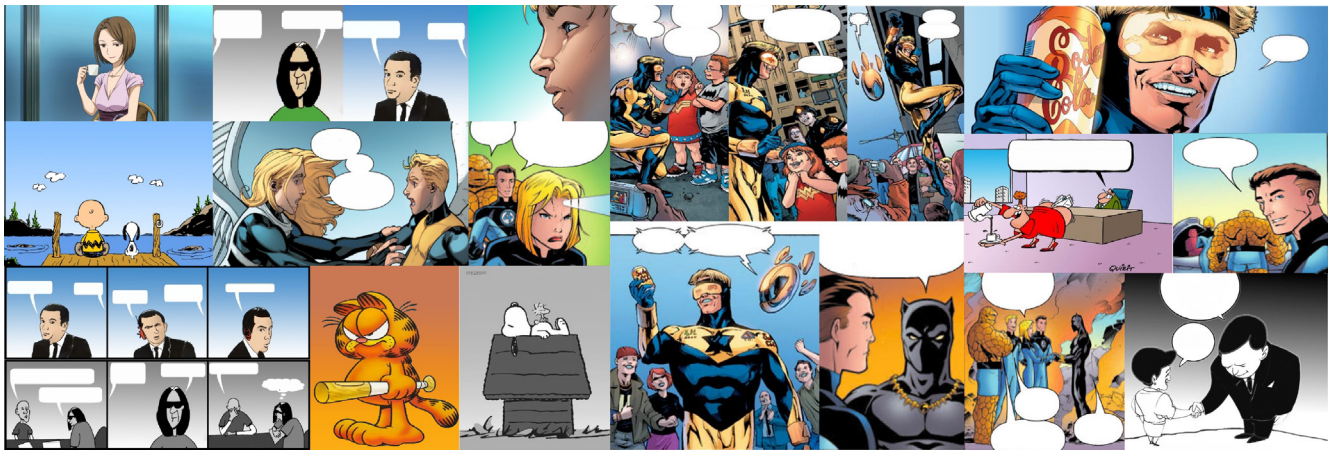


Fig. 11. Partial comic images from dataset are used as our test data.

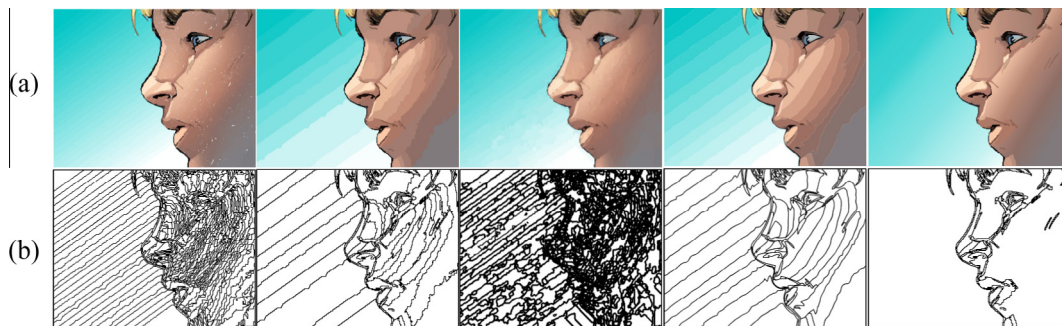


Fig. 12. The SVG output results of the test comic image #4 by using (from left to right) Autotrace, Autotrace+Merge, SwaterG, Vector Magic and our method. (a) The rendering images. (b) The region boundaries. Our method can produce clear boundary with fewer regions.

Case 3 (The intersection points > 1 and the points of $path_i$ are all enclosed by the $path_j$):

Fig. 7(c) (top) shows that, considering the next point of C, both candidates E and D appear in both paths. By tracking their next points F and G respectively ($E \rightarrow F$, $D \rightarrow \{F, G\}$), we choose point G as the next point because it appears only in one path. Then, the point D is added as the next point as it's the previous point of G. Fig. 7(c) (bottom) shows that two paths are merged when the IVCS procedure goes back to point A ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow G \rightarrow H \rightarrow I \rightarrow J \rightarrow A$). Path 2 can be removed. Moreover, points C and D can be removed for compression by the post-processing method described in Section 3.5.

Case 4 (The intersection points = 1):

Fig. 7(d) (top) shows that only point C appears in both two paths. Points (E, F, G, C) in path 2 can be added to path 1. Then, the point D is added as the next point. Fig. 7(d) (bottom) shows that two paths are merged when the IVCS procedure goes back to point A ($A \rightarrow B \rightarrow C \rightarrow E \rightarrow F \rightarrow G \rightarrow C \rightarrow D \rightarrow H \rightarrow I \rightarrow A$).

3.5. Post-processing

As shown in Fig. 7(c) (bottom), we observe that some redundancy points in paths are not removed. Moreover, as the example shown in Fig. 8(a), some concave and overlapped paths may need



Fig. 13. Comparing vectorization results of more comic images. (a) The test comic image #7. (b) The test comic image #16. (c) The test comic image #6. The methods applied from left to right and top to down are DoNothing (original comic image), Autotrace, Autotrace+Merge, SwaterG, Vector Magic, and Our method.

to be re-organized for better compression. In this paper, we design a post-processing scheme with path simplification and W3C simplification to further compress SVG files. Fig. 8(b) shows an example of paths those are re-organized and optimal encoded after our post-processing scheme. Our path simplification scheme has two steps. At the first step, we decide the layer order of two neighboring paths. Then, we try to represent the layer covered by another layer with fewer points. Fig. 9 shows an example where points belong to path P_i following clockwise direction are also in path P_j following counter-clockwise direction. According to data characteristics in Autotrace, P_i and P_j are decided to be bottom and upper layer respectively. We can apply a straight line to replace some points in the concave path P_i . Note that this process will be executed when there are at least three points in neighboring paths. According to the suggestions of W3C [18], we implement some major SVG simplifying methods in our system including using “h” (“v”) for representing “horizontal” (“vertical”), combining “l” which have same slope, and eliminating extraneous white space and separators.

3.6. Linear gradient approximation

SVG linear gradient is used to fill-in enclosed path to approximate color gradients in original comic images. Before filling the gradients, some elements should be determined. We define gradients by using `<linearGradient>` elements and giving each of them a unique id via its “id” attribute. For example, `<linearGradient id = “1”>...</linearGradient>`. The attributes of `<x1, y1>` and `<x2, y2>` describe starting point and ending point of the CG direction, respectively. The attributes of `<offset>` and `<stop-color>` represent the color transition at specific locations along the gradient vector that can be obtained by recording colors and calculating area percentages of each path. The path then references the linear gradient as the value of the “fill:url” attribute such as `<path style = “fill:url(#1)”...>`. We use the attributes to explain how to fill the linear gradient in the left side of Fig. 10. Due to CGV is 8, the attributes of `<x1, y1>` and `<x2, y2>` are equal to `<0%, 0%>` and `<100%, 100%>` respectively. The `<offset>` from “4%” to “100%” represents the `<stop-color>` is smoothly transiting from “#07C6C6” to “#F8FEFE”.

The linear gradient syntax is then generated as shown in the right side of Fig. 10. Moreover, our method is explained in Algorithm 1. Fig. 6(d) shows the approximation gradient result in gradient layer by embedding the linear gradient syntax. Our results proved that using eight-direction to represent gradient regions are adequate for producing visually acceptable results.

Algorithm 1 Linear gradient approximation

```

1: For each IVCS region  $IVCS_j$  // (e.g. as shown in Fig. 6(c))
2:   define tag <linearGradient> and determine values of
   attributes <id>, <x1, y1> and <x2, y2>
3:   For each region  $R_i \in IVCS_j$  // (e.g. as shown in Fig. 6(a))
4:     define the tag <stop> and determine the values of
     attributes <offset> and <stop-color>
5:      $offset \leftarrow R_i.area / IVCS_j.area$  // Calculate the area
     percentage
6:      $stop-color \leftarrow \text{convert } R_i.rgb \text{ to } R_i.hex$ 
7:   End For
8: <path style = “fill:url(#id)”...> // Reference the linear
   gradient
9: End For

```

4. Experimental results

In this paper, without loss of generality, we use 130 comic images collected from the Internet as our test data, part of them (21 comic images) are as shown in Fig. 11. Our test comics used for comparison are of different kinds: single color frame, gray levels frame, and single tier. All of them are originally compressed by the JPEG format. In order to evaluate the performance of the proposed method, we compare our results to those produced by other recent vectorization techniques, including Autotrace [11], Autotrace+Merge [1], SWaterG [15] and Vector Magic [9]. To make sure the fairness of our experiments, the same noise-reducing filter has been applied to all test images. Furthermore, we adopt settings to preserve most details of comic images in Autotrace and Vector Magic. Parameter setting of Autotrace+Merge and SWaterG are as

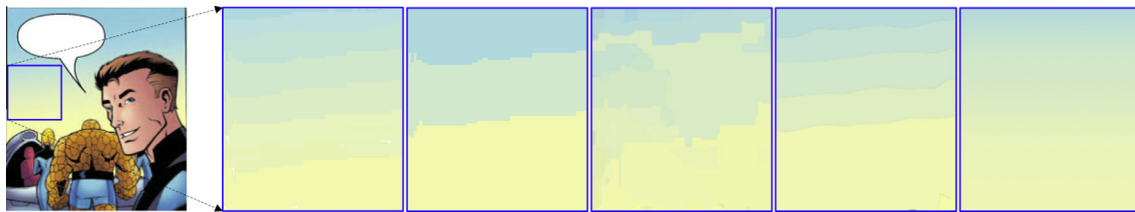


Fig. 14. Comparing vectorization results of the gradient detail in the blue box of the test comic image #21. The methods applied from left to right and top to down are DoNothing (original comic image), Autotrace, Autotrace+Merge, SWaterG, Vector Magic, and Our method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)



Fig. 15. The test comic image #9 demonstrates the utilization of the reusability of SVG syntax.

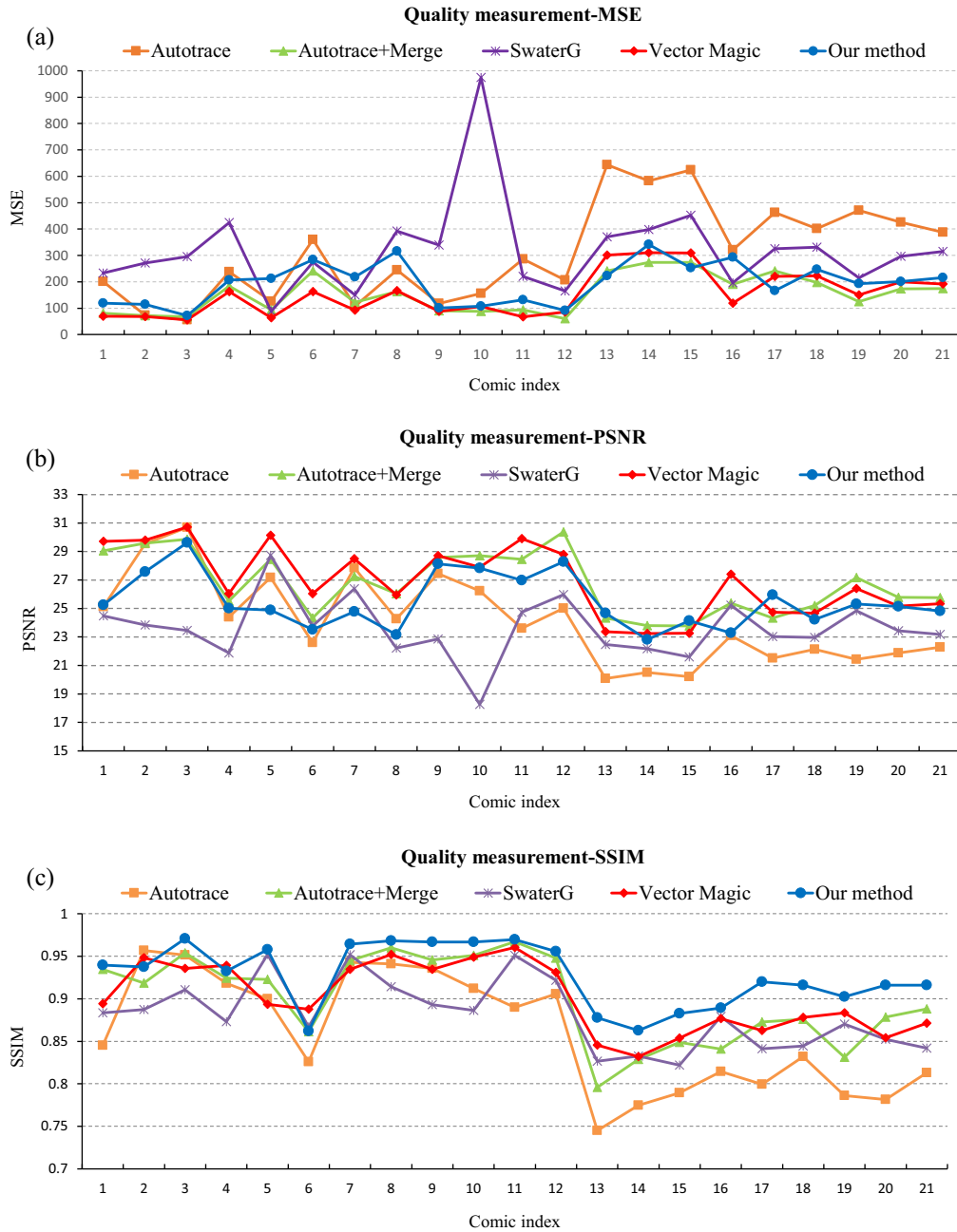


Fig. 16. Comparison with other methods in terms of (a) MSE, (b) PSNR and (c) SSIM.

described in [1,15]. The metrics of our experimental results include quality measurement, size measurement, and rendering time on handheld device. Note in quality measurement, we rasterize all vector output images by applying batik-SVG rasterizer [19] and then compare it with the original images.

4.1. Comparison

Fig. 12 shows that our results can produce clear boundary with fewer regions. Our results give almost the same perceptual quality to the original comic images than the ones generated by the other methods especially in regions with CG. We could observe Autotrace always produce scrappy areas and a snow effect caused by the presence of gaps between near paths. The other three methods generated lots of single color regions due to the change of CG. Note that SWaterG generates extra black noises around edge. These

results not only degrade visual quality, but also expand the file size. More comparisons of the SVG output results are given in Fig. 13.

Fig. 14 shows that we magnify comic images by a factor of 3 to observe the detail in subpart images (blue box). The result shows that our method can approximate linear gradient from original comic images by embedding the linear gradient syntax to provide good perceptual quality.

We observe that, in real comic works, there are many regions with the same or similar gradient vectors. For example, as our test comic image #9 shown in Fig. 15, it has three similar backgrounds. To achieve reusable functionality, these paths can reference the same *<linearGradient>* element by setting on the same id via its id attribute. Therefore, the reusability of SVG syntax causes SVG file size reducing 1016 bytes in this case. The more gradients can be reused, the smaller file size we can achieve.

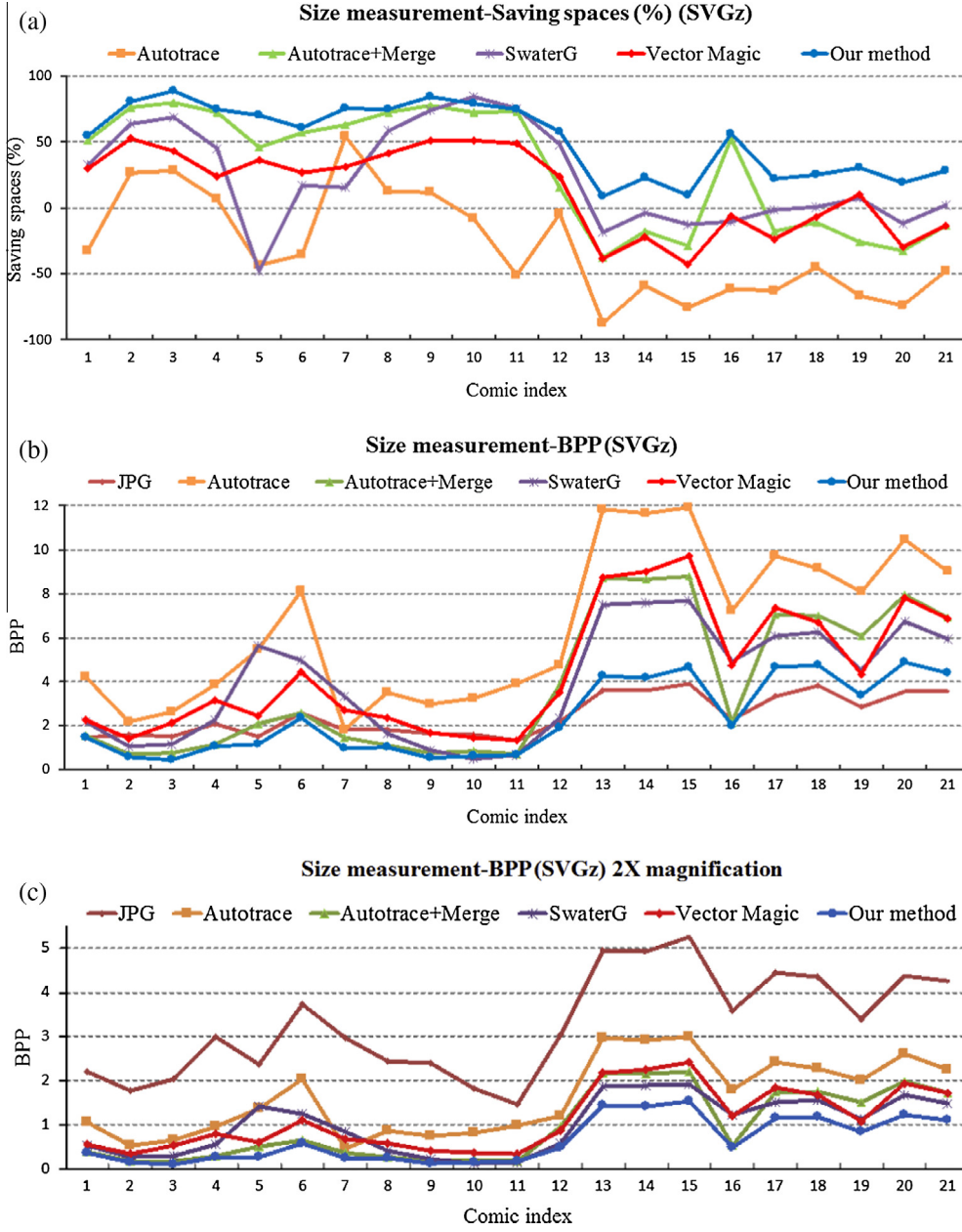


Fig. 17. Comparison with other methods in terms of (a) saving spaces, (b) BPP and (c) BPP (2× magnification).

4.1.1. Quality measurement

We use three well know objective metrics, MSE (mean square error), PSNR (peak signal-to-noise ratio) and SSIM (structural similarity index metric) in image quality measurement. MSE is the average square difference between a reference image and a compressed image. The equation can be expressed as below.

$$MSE = \frac{1}{H \times W} \sum_{m=1}^H \sum_{n=1}^W [\delta(m, n)]^2 \quad (7)$$

$$\delta(m, n) = [f(m, n) - g(m, n)] \quad (8)$$

where H and W are height and width of the image, $f(m, n)$ and $g(m, n)$ are referenced image and compressed image. PSNR can be measured via MSE in decibel (dB). It is defined as follows.

$$PSNR = 10 \log_{10} \left(\frac{255^2}{MSE} \right) \quad (9)$$

Usually the lower MSE or higher PSNR value provides more similarity of two images in pixel geometry distance. However, in Fig. 16 (a) and (b), we observe that our method does not always show the best MSE and PSNR even our results are good in the quality perception of the human visual system (HVS). One reason is that embedding the linear gradient syntax to represent the continuous CG may have slight difference from original comics. The other is these metrics cannot actually reflect image detail preserved levels [20,21]. Comparison with MSE and PSNR, SSIM reflects image degradation as perceived change in structural information. It is considered to be correlated with the quality perception of HVS.

$$SSIM(f, g) = \frac{(2\mu_f\mu_g + c_1)(2\sigma_{fg} + c_2)}{(\mu_f^2 + \mu_g^2 + c_1)(\sigma_f^2 + \sigma_g^2 + c_2)} \quad (10)$$

where $\mu_f, \mu_g, \sigma_f^2, \sigma_g^2$, and σ_{fg} are the average, variance and covariance of f and g , c_1 and c_2 are two variables to stabilize the division with weak denominator.

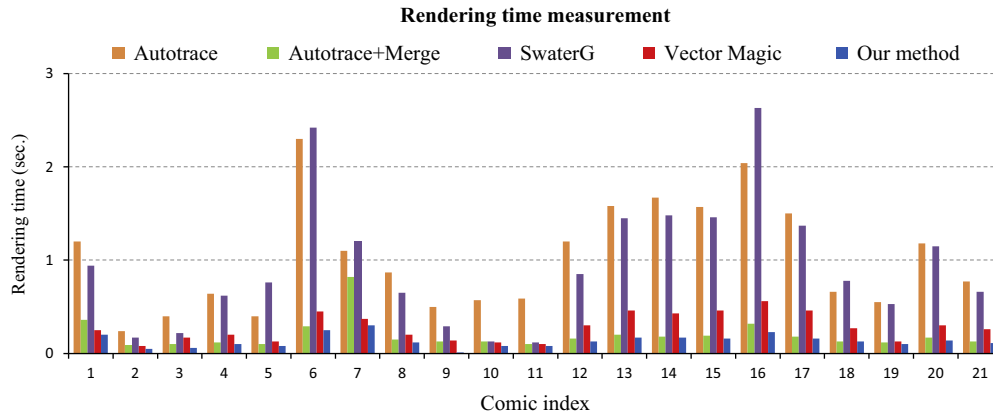


Fig. 18. Comparison with other methods in terms of rendering time on ASUS Padfone-2 with Qualcomm Snapdragon 1.5 GHz processor and 2 GB RAM.

Table 1
Quantitative experimental results.

	Quality measurement			Size measurement		Rendering time measurement (s)
	MSE	PSNR	SSIM	Saving spaces (%)	BPP	
JPG	–	–	–	–	3.35	–
Autotrace	312.18	23.19	0.85	–35.42	6.85	1.12
Autotrace+Merge	153.50	26.27	0.92	33.49	3.89	0.25
SwaterG	325.62	23.00	0.88	20.25	4.15	1.01
Vector magic	152.68	26.29	0.91	12.50	4.53	0.30
Our method	192.71	25.28	0.94	57.23	2.42	0.12

Fig. 16(c) shows that our method's SSIM outperforms in most cases of others. But under some conditions, there are some cases perform better than our method. We observe comic #2 in Autotrace generates lots of thin regions in CG. This result maintains certain perceptual quality without scaled up in size, but it pays the penalty for size measurement as shown in Fig. 17. Moreover, comic #4 and comic #6 in Vector Magic produce more smooth edge than our method. The averages SSIM are 0.86 by Autotrace, 0.9 by Autotrace+Merge, 0.88 by SWaterG, 0.9 by Vector Magic and 0.93 by our method.

4.1.2. Size measurement

In size measurement, we calculate both space savings and bit per pixels (BPP) between the SVG outputs files and original comic images. Note that, in real world applications, the system software G-zip is applied to compress the text-mode SVG file to binary-mode SVGz format. However, reducing original SVG file size is still the foundation for achieving better results. Let $|X|$ denotes the size of file X , this paper defines the space savings and BPP as follows.

$$\text{Space savings}_{\text{SVGz}}(\%) = \frac{(|\text{Original raster image}| - |\text{SVGz file}|)}{|\text{Original raster image}|} \times 100 \quad (11)$$

$$\text{BPP}_{\text{SVGz}} = \frac{|\text{SVGz file}|}{(\text{Width of original raster image} \times \text{Height of original raster image}/8)} \quad (12)$$

Fig. 17(a) shows our method can save about 52.13% in average space saving, outperforms the other methods (such as –29.38% by Autotrace, 29.57% by Autotrace+Merge, 23.09% by SwaterG, and 13.5% by Vector Magic). This is because our method produce meaningful and fewer regions than other methods. However, SwaterG in comic #10 saves about 84.3%, better than our method (about 79.1%) due to it produces large and flat regions in CG. The average BPP achieved by our method is about 2.37 in Fig. 17(b). It is smaller than 2.45 (JPG), 6.46 (Autotrace), 3.84 (Autotrace+Merge), 3.99 (SwaterG) and 4.48 (Vector Magic). We observe that the BPP of JPG is

smaller than our method if the image resolution is really low. For enlarging comic images as reasonable resolutions (e.g. $2\times$ magnification), the JPG version more quickly escalates in size as shown in Fig. 17(c).

4.1.3. Rendering time measurement

Modern mobile devices have built the SVG support in browser. Due to the limited CPU speed and memory space, SVG rendering time on mobile devices become one of the major problems. In this paper, we use ASUS Padfone-2 as our test mobile device which runs Android 4.1.1 OS on Qualcomm Snapdragon 1.5 GHz processor with 2 GB RAM. As there is no speed test tool specific for SVG rendering, a timer script is written on chrome browser to count its rendering time. Each comic is rendered three times and then average them. Our average rendering time is about 0.13 s. It is much less than the other methods as shown in Fig. 18. The more points included in a vector image will spend more rendering time. Besides, the scrappy color regions will also increase rendering time.

To further validate the accuracy of the proposed algorithm, we have tested our algorithm by a larger dataset with 130 comic images. Quantitative results of our experiments are shown in detail in Table 1.

4.2. Extensions

Unlike linear gradient, radial gradients spread CG in a circular pattern from a focus point to its ending shape. They might have a higher probability in all eight directions. Our method to process radial gradients is demonstrated. Fig. 19(a) is original comic image. Fig. 19(b) shows the CGV result (red¹ arrows indicate the direction of the gradient). Fig. 19(c) shows the final result. As SVG also pro-

¹ For interpretation of color in Fig. 19, the reader is referred to the web version of this article.

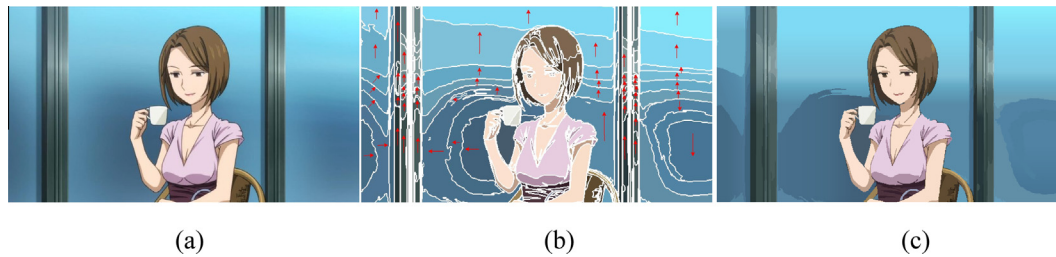


Fig. 19. The original CGV fails to identify the radial type of gradient. (a) The original comic image. (b) The CGV result. (c) The final result. As SVG also provides the radial type syntax of gradient, the proposed method can be extended to cope with comic images with radial gradients.

vides the radial type syntax of the gradient, the proposed method can be extended to cope with comic images with radial gradients.

5. Conclusion

Raster images lack scalable property and must adjust file scales for different sizes of display devices. HTML5 with SVG vector images will become future media in the Internet. Even some approaches have provided vector converting functions; they still cannot meet the need of image quality, compression ratio, and rendering speed. In this paper, we propose CGV combined with IVCS by embedding SVG linear gradients to deal with the CG problem of comic images. We also introduced post-processing consisted of some simplification rules to eliminate redundancy points. Experimental results show our method have better perceptual quality, smaller file size and faster rendering speed than the previous methods. In our future works, we intend to extend our current approaches to consider radial gradients.

References

- [1] Ray-I. Chang, Yachik Yen, Ting-Yu Hsu, An XML-based comic image compression, *Lect. Notes Comput. Sci.* 5353 (2008) 563–572.
- [2] Marvel. <<http://marvel.com/>>.
- [3] ITMedia eBook. <<http://ebook.itmedia.co.jp/ebook/articles/1306/27/news088.html>>.
- [4] SVG. <<http://www.w3.org/Graphics/SVG/>>.
- [5] Song-Hai Zhang, Tao Chen, Yi-Fei Zhang, Hu. Shi-Min, R.R. Martin, Vectorizing cartoon animations, *IEEE Trans. Visual Comput. Graphics* 15 (2009) 618–629.
- [6] Kei Kawamura, Hiroshi Watanabe, Hideyoshi Tominaga, Vector representation of binary images containing halftone dots, in: *IEEE International Conference on Multimedia and Expo*, 2004, pp. 335–338.
- [7] Kei Kawamura, Yuki Yamamoto, Hiroshi Watanabe, Gradation approximation for vector based compression of comic images, in: *IEEE International Conference on Image Processing*, 2005, pp. 11–14.
- [8] S. Battiato, G. Gallo, G. Messina, SVG rendering of real images using data dependent triangulation, in: *Proc. of ACM/SCCG*, 2004, pp. 191–198.
- [9] Vector Magic. <<http://vectormagic.com/home>>.
- [10] Adobe SVG Optimizing. <<http://www.adobe.com/svg/workflow/optimizing.html>>.
- [11] Autotrace. <<http://autotrace.sourceforge.net/>>.
- [12] Qu Yingge, Tien-Tsin Wong, Pheng-Ann Heng, Manga colorization, *ACM Trans. Graph.* 25 (2006) 1214–1220.
- [13] Daniel Šýkora, Jan Buriánek, Jiří Žára, Unsupervised colorization of black-and-white cartoons, in: *International Symposium on Non-Photorealistic Animation and Rendering*, 2004, pp. 121–127.
- [14] Daniel Šýkora, Jan Buriánek, Jiří Žára, Colorization of black-and-white cartoons, *Image Vision Comput.* (2005) 767–852.
- [15] S. Battiato, A. Costanzo, G. Di Blasi, G. Gallo, S. Nicotra, SVG rendering by watershed decomposition, *The International Society for Optical Engineering*, 2005, pp. 23–32.
- [16] Tomasi Carlo, Roberto Manduchi, Bilateral filtering for gray and color images, in: *IEEE International Conference on Computer Vision*, 1998, pp. 839–846.
- [17] Hsin-Chia Chen, Sheng-Jyh Wang, The use of visible color difference in the quantitative evaluation of color image segmentation, in: *IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 3, 2004, pp. 593–596.
- [18] Minimizing SVG File Sizes. <<http://www.w3.org/TR/SVG11/minimize.html>>.
- [19] Apache™ batik SVG toolkit. <<https://xmlgraphics.apache.org/batik/>>.
- [20] Weisi Lin, C.-C. Jay Kuo, Perceptual visual quality metrics: a survey, *J. Vis. Commun. Image Represent.* 22 (2011) 297–312.
- [21] Ya-Hui Shiao, Tzong-Jer Chen, Keh-Shih Chuang, Cheng-Hsun Lin, Chun-Chao Chuang, Quality of compressed medical images, *J. Dig. Imag.* 20 (2007) 149–159.
- [22] Jian Sun, Lin Liang, Fang Wen, Heung-Yeung Shum, Image vectorization using optimized gradient meshes, in: *ACM Transactions on Graphics (TOG)*, vol. 26, 2007, pp. 11.
- [23] Gregory Lecot, Bruno Levy, Ardeco: Automatic Region DEtection and COnversion, in: *Eurographics Symposium on Rendering*, 2006, pp. 349–360.
- [24] Ruchin Kansal, Subodh Kumar, A vectorization framework for constant and linear gradient filled regions, *Visual Comput.* 31 (2015) 717–732.